

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY  
INTERNATIONAL UNIVERSITY  
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**Net-Centric Programming  
Mangahub Project Report**

**Authors**

- Lê Đoàn Minh Ngọc – ITITDK23023
- Phạm Trung Kiên – ITCSIU23020

**Instructor: Lê Thanh Sơn – Nguyễn Trung Nghĩa**

**Course: Net-centric Programming – IT096IU**

**Programming Language: Go**

**Team Size: 2 students**

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Project Overview</b>                         | <b>3</b>  |
| <b>2. System Objectives (Learning Focus)</b>       | <b>3</b>  |
| <b>3. System Architecture Overview</b>             | <b>4</b>  |
| <b>4. Data Models</b>                              | <b>4</b>  |
| <b>5. Database Schema (SQLite)</b>                 | <b>5</b>  |
| <b>6. HTTP REST API Documentation</b>              | <b>6</b>  |
| 6.1 HTTP REST API                                  | 6         |
| 6.2 Health Check                                   | 6         |
| 6.3 User Registration                              | 6         |
| 6.4 User Login                                     | 7         |
| 6.5 Search Manga                                   | 7         |
| 6.6 Get Manga Detail                               | 8         |
| 6.7 Add Manga to User Library                      | 8         |
| 6.8 Get User Library                               | 9         |
| <b>7. TCP Progress Synchronization (Port 9090)</b> | <b>9</b>  |
| <b>8. UDP Notification System (Port 9091)</b>      | <b>10</b> |
| <b>9. WebSocket Chat (Port 9093)</b>               | <b>10</b> |
| <b>10. gRPC Manga Service (Port 9092)</b>          | <b>10</b> |
| <b>11. Setup Instructions</b>                      | <b>11</b> |
| <b>12. CLI Usage</b>                               | <b>12</b> |
| <b>13. Architecture Overview</b>                   | <b>13</b> |

# 1. Project Overview

MangaHub is a manga and comic tracking system developed in Go for the Net-centric Programming course. The main purpose of the project is to demonstrate how different network communication protocols can work together inside one complete application.

The system allows users to:

- Register and log in
- Search manga and view detailed information
- Manage a personal library (reading, completed, plan-to-read, on-hold, dropped)
- Update and synchronize reading progress in real time
- Receive lightweight chapter notifications
- Chat with other users in real time
- Communicate between internal services using gRPC

MangaHub combines the following protocols:

- HTTP REST API for standard request/response operations
- TCP for real-time progress synchronization
- UDP for lightweight broadcast notifications
- WebSocket for live chat
- gRPC for internal service communication

The project stores data in a SQLite database. User accounts, manga information, and reading progress are saved persistently. Passwords are protected using bcrypt hashing, and authenticated HTTP endpoints use JWT tokens.

Overall, MangaHub is not only a simple manga tracking application, but also a practical demonstration of network programming concepts, concurrent server execution, database integration, and service communication in Go.

## 2. System Objectives (Learning Focus)

**This project is designed to help students:**

- Apply core networking concepts in a practical system
- Practice building services with multiple protocols
- Understand concurrent programming in Go (goroutines + channels)
- Integrate a shared database across services
- Build a realistic, testable application within academic constraints

### 3. System Architecture Overview

The architecture is modular and protocol-based. Each communication channel has its own server and responsibilities, but all share the same SQLite database.

#### 3.1 Core Components

##### 1) HTTP REST API (Port 8080)

- Handles authentication, manga search, library management, and progress updates.

##### 2) TCP Progress Sync (Port 9090)

- Broadcasts reading progress updates to connected clients in real time.

##### 3) UDP Notification Server (Port 9091)

- Sends lightweight chapter release notifications to subscribed users.

##### 4) gRPC Manga Service (Port 9092)

- Provides internal service calls for manga search/detail and progress updates.

##### 5) WebSocket Chat Server (Port 9093)

- Enables real-time chat and community discussions.

#### 3.2 Shared Data Layer

All services share a SQLite database:

- users
- manga
- user\_progress

This design ensures that updates from any protocol are consistent across the system.

### 4. Data Models

Manga

```
{  
  "id": "one-piece",  
  "title": "One Piece",  
  "author": "Oda Eiichiro",  
  "genres": ["Action", "Adventure", "Shounen"],  
  "status": "ongoing",  
  "total_chapters": 1100,  
  "description": "A young pirate's adventure...",  
  "cover_url": "https://example.com/covers/one-piece.jpg"
```

```
}
```

User Progress

```
{
  "user_id": "user123",
  "username": "manga_lover",
  "reading_lists": {
    "reading": [
      {
        "manga_id": "one-piece",
        "current_chapter": 1095,
        "status": "reading",
        "last_updated": "2024-01-20T10:30:00Z"
      }
    ],
    "completed": [],
    "plan_to_read": []
  }
}
```

## 5. Database Schema (SQLite)

```
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  username TEXT UNIQUE,
  password_hash TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE manga (
  id TEXT PRIMARY KEY,
  title TEXT,
  author TEXT,
  genres TEXT,      -- JSON array as text
  status TEXT,
  total_chapters INTEGER,
  description TEXT
);
```

```
CREATE TABLE user_progress (
  user_id TEXT,
```

```
manga_id TEXT,  
current_chapter INTEGER,  
status TEXT,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (user_id, manga_id)  
);
```

## 6. HTTP REST API Documentation

### 6.1 HTTP REST API

The HTTP REST API runs on:

**http://localhost:8080**

It is used for user authentication, manga browsing, library management, and reading progress updates.

### 6.2 Health Check

#### GET /ping

This endpoint checks whether the HTTP server is running.

#### Example response:

```
{  
  "message": "pong"  
}
```

### 6.3 User Registration

#### POST /auth/register

This endpoint creates a new user account.

#### Request body:

```
{  
  "username": "alice",  
  "email": "alice@example.com",  
  "password": "password123"
```

```
}
```

**Successful response:**

```
{  
  "message": "Account created successfully",  
  "user_id": "usr_...",  
  "username": "alice",  
  "email": "alice@example.com"  
}
```

## 6.4 User Login

**POST /auth/login**

This endpoint authenticates a user and returns a JWT token.

**Request body:**

```
{  
  "username": "alice",  
  "password": "password123"  
}
```

**Successful response:**

```
{  
  "message": "Login successful",  
  "token": "jwt_token_here",  
  "username": "alice",  
  "user_id": "usr_..."  
}
```

The returned token must be used for protected user endpoints.

## 6.5 Search Manga

**GET /manga**

This endpoint searches manga in the database.

**Example:**

GET /manga?query=One

**Optional filters include:**

query  
genre  
status

**Example response:**

```
{
  "count": 3,
  "results": [
    {
      "id": "one-piece",
      "title": "One Piece",
      "author": "Eiichiro Oda",
      "genres": ["Action", "Adventure", "Shounen"],
      "status": "ongoing",
      "total_chapters": 1110,
      "description": "A pirate adventure following Luffy and the Straw Hat crew."
    }
  ]
}
```

## 6.6 Get Manga Detail

### **GET /manga/{id}**

This endpoint returns detailed information about a specific manga.

**Example:**

GET /manga/one-piece

## 6.7 Add Manga to User Library

### **POST /users/library**

This is a protected endpoint. It requires a JWT token.

**Header:**

**Authorization:** Bearer <token>

Request body:

```
{
  "manga_id": "one-piece",
  "current_chapter": 1,
  "status": "reading"
}
```



Successful response:

```
{  
  "message": "manga added to library"  
}
```

## 6.8 Get User Library

### GET /users/library

This endpoint returns the manga list saved in the current user's library.

Header:

Authorization: Bearer <token>

Update Reading Progress

PUT /users/progress

This endpoint updates the current reading progress of a manga.

Header:

Authorization: Bearer <token>

Request body:

```
{  
  "manga_id": "one-piece",  
  "current_chapter": 50,  
  "status": "reading"  
}
```

Successful response:

```
{  
  "message": "progress updated"  
}
```

## 7. TCP Progress Synchronization (Port 9090)

The TCP server runs on:

localhost:9090

It is used to broadcast reading progress updates to connected clients.

Example CLI commands:

```
.\mangahub.exe sync connect
```

```
.\mangahub.exe progress update --manga-id one-piece --chapter 51
```

The message format is simple:

```
user_id|manga_id|chapter
```

## 8. UDP Notification System (Port 9091)

The UDP server runs on:

localhost:9091

It is used to send chapter release notifications.

Example CLI commands:

```
.\mangahub.exe notifications listen --username alice
```

```
.\mangahub.exe notifications send --title "One Piece" --chapter 1111
```

## 9. WebSocket Chat (Port 9093)

The WebSocket server runs on:

ws://localhost:9093/ws

It is used for real-time chat between users.

Example CLI commands:

```
.\mangahub.exe chat --username alice
```

```
.\mangahub.exe chat --username bob
```

## 10. gRPC Manga Service (Port 9092)

The gRPC server runs on:

localhost:9092

Service:

```
service MangaService {  
    rpc GetManga(GetMangaRequest) returns (MangaResponse);  
    rpc SearchManga(SearchRequest) returns (SearchResponse);  
    rpc UpdateProgress(ProgressRequest) returns (ProgressResponse);  
}
```

```
.\mangahub.exe manga search --query One  
.\mangahub.exe manga get --id one-piece  
.\mangahub.exe manga grpc-progress --user-id usr_123 --manga-id one-piece --chapter  
52 --status reading
```

## 11. Setup Instructions

### Requirements

- Go (1.19+)
- PowerShell or terminal
- Git (if cloned from repository)

### Main Libraries

- Gin (HTTP API)
- Cobra (CLI)
- Gorilla WebSocket
- gRPC + Protocol Buffers
- SQLite
- JWT
- bcrypt

### Step 1: Open the Project Folder

Open PowerShell and move to the project directory:

```
cd "C:\STUDY\Net centric\Project\mangahub"
```

### Step 2: Download Dependencies

If needed, run:

```
go mod tidy
```

### Step 3: Build the Application

Build the executable file:

```
go build -o mangahub.exe .\cmd\main
```

After this step, the file mangahub.exe should be available in the project folder.

#### **Step 4: Start All Servers**

Run:

```
.\mangahub.exe server start
```

This starts:

- HTTP (8080)
- TCP (9090)
- UDP (9091)
- gRPC (9092)
- WebSocket (9093)

The terminal running this command should remain open during the whole demo or testing session.

#### **Step 5: Test the HTTP Server**

Open another terminal and run:

```
Invoke-RestMethod http://localhost:8080/ping
```

If the server is working, it returns:

```
pong
```

## **12. CLI Usage**

After the server is running, the CLI can be used to test different features.

#### **Register:**

```
mangahub auth register --username myuser --email myuser@example.com
```

#### **Login:**

```
mangahub auth login --username myuser
```

#### **Search:**

```
mangahub manga search "one piece"
```

#### **Library Add:**

```
mangahub library add --manga-id one-piece --status reading
```

**Update Progress:**

mangahub progress update --manga-id one-piece --chapter 1095

**For TCP sync:**

.\mangahub.exe sync connect

**For UDP notification:**

.\mangahub.exe notifications listen --username alice

**For WebSocket chat:**

.\mangahub.exe chat --username alice

**For gRPC manga search:**

.\mangahub.exe manga search --query One

## 13. Architecture Overview

MangaHub follows a modular and multi-protocol architecture. Each main function is separated into its own package, making the system easier to understand, maintain, and extend.

At the center of the system is the command-line interface. The CLI is built with Cobra and acts as the main entry point for users. Through CLI commands, users can start servers, connect to TCP sync, listen for UDP notifications, join WebSocket chat, and call gRPC manga services.

The backend is composed of five communication services.

The HTTP REST API handles traditional request-response operations. It is responsible for authentication, manga search, manga details, library management, and reading progress updates. It uses JSON for request and response data.

The TCP server is responsible for progress synchronization. It accepts multiple client connections and broadcasts reading progress updates in real time. This demonstrates persistent socket communication and concurrent connection handling with goroutines.

The UDP server is responsible for notifications. Clients register with the server, and the server broadcasts chapter release messages to registered clients. This demonstrates lightweight, connectionless communication.

The WebSocket server provides real-time chat. It keeps an open two-way connection between clients and the server, allowing users to send and receive messages instantly.

The gRPC server works as an internal manga service. It uses Protocol Buffers to define clear service methods and message structures. It supports manga search, manga detail retrieval, and progress updates.

All services share the same SQLite database. The database stores users, manga information, and user reading progress. This shared persistence layer allows the different protocols to work together as one system instead of separate small programs.